Ivan Hinojos

Homework 4

GenAI: Claude

I asked Claude to generate me 3 essay questions after uploading the slides, so Claude knows the context of what I'm asking and it gave me this:

1. Explain why a deep network with several hidden layers can represent certain functions far more efficiently than a shallow network with one wide hidden layer. Use the idea of composing piecewise linear functions to support your answer.
2. Starting from the idea of a model that predicts a probability distribution over outputs, explain how the maximum likelihood principle leads to the least squares loss for regression and the cross-entropy loss for classification. State what assumption about the output distribution each loss makes.
3. Compare a fully connected layer with a convolutional layer for image data. Discuss parameter count, translation equivariance, and why convolutional networks suit images better.

My answers to the questions:

1. A network having several layers can be much better at representing certain functions compared to a shallow network because each layer will have access to the output produced by the preceding layer. Neural networks commonly employ piecewise linear functions like ReLU, which break down the input domain into several linear subdomains. It takes a lot of neurons to generate many subdomains for a shallow network. But for a deep network, piecewise linear functions can be composed over several layers, resulting in an exponential growth in the number of subdomains and complexity of the function being generated.

2. A model may also be seen to predict a distribution over its outputs. The maximum likelihood method picks out those model parameters that make the training examples most probable. In regression, if one assumes that the target values are normally distributed about the predictions of the model, then maximizing the

likelihood leads to the least squares objective function that computes the squared error between the prediction and the true value. In classification, if one assumes a categorical distribution for the outputs, maximizing the likelihood leads to the cross entropy objective function.

3. The completely connected layer is a layer where each neuron connects to each input, which might require an enormous number of parameters in case of handling images. The convolutional layer applies filters to various parts of the image and hence requires fewer parameters compared to the completely connected layer. In addition, the convolutional layers have the property of translational equivariance; hence, if an object moves within the image, the features found by the filters also move. Hence the convolutional neural networks are very appropriate for images as they will detect the same kind of features like edges and shapes wherever they are in the images.

I asked Claude to judge my answer and I got this:

These are solid answers. All three are technically correct, and the main weakness is that they stay a little general. Here's my critique for each one.

Q1: Deep vs. shallow networks (strong, but missing specifics)

- You never give a concrete example. A shallow network with D hidden units makes at most D+1 linear regions for a 1D input. A deep network can multiply regions layer by layer, so the count grows much faster than the parameter count. Even one number like this would make your argument much more convincing.
- The "folding" intuition is the heart of why composition helps: each layer folds the input space so later layers reuse the same structure repeatedly.
- You imply deep is always better. Say that this holds for certain function families, and that shallow networks can still approximate any function given enough units (the universal approximation theorem). That makes the contrast more accurate.
- Briefly acknowledge the tradeoff: deeper networks are harder to train.

Q2: Maximum likelihood and losses (correct, but missing some mechanics)

- Say that in practice we minimize the *negative log-likelihood*, and explain why the log is used (products become sums and it's numerically stable).

- State the assumptions explicitly: training examples are independent and identically distributed, and the Gaussian has a fixed variance, with the model output as the mean. That fixed variance is what makes the loss reduce to plain squared error.
- For classification, mention that a softmax turns the network's outputs into valid probabilities. Cross-entropy is also more precisely the negative log of the probability given to the correct class, and you could add that binary classification uses a Bernoulli distribution.

Q3: Fully connected vs. convolutional (good, but conflates two ideas)

- You name the benefits but not the mechanisms. Add **weight sharing** (the same filter reused at every position) and **local connectivity** (each unit sees only a small patch).
- The last sentence of your second-to-last point blurs equivariance and invariance. Convolution is equivariant, meaning shifts in the input shift the output. Pooling or later layers are what give approximate invariance. Keep them distinct.
- Add a concrete parameter comparison. For example, a fully connected layer on a 224×224×3 image needs millions of weights per output unit, while a 3×3 filter needs only 27.
- You could also mention that stacking layers grows the receptive field, so deeper convolutional layers capture larger patterns.

Overall: accurate, clear, and well organized. Adding one concrete number or example per answer and stating assumptions explicitly would take them from good to excellent.

Then I updated my answer to this:

1. A deep neural network with multiple hidden layers can provide a more efficient representation of certain complex functions compared to a shallow network with one hidden layer and large width. It is since each additional layer can capitalize on what was learned by the previous layer. With piece-wise linear activation functions, such as ReLU, each additional layer can form its own linear segments. For instance, a shallow neural network with (D) hidden neurons can partition a one-dimensional space into not more than (D+1) linear segments. As the result of the composition, a deep network can form significantly more of them. Intuitively, it means that a deep network “folds” the input space to let the next layer take advantage of this and

construct something more complex. However, a shallow network with sufficiently many neurons can represent any function.

2. The model itself can be viewed as the model that predicts some probability distribution over its outputs. Maximum likelihood aims at finding such parameters of the model that would maximize the likelihood of training samples. In practice, it is more convenient to minimize the negative log-likelihood since applying logarithm converts the product of probabilities to their sum. The assumption about the distribution of target variables in regression is that it is Gaussian with the expected value predicted by the model and the fixed variance. With this assumption, the negative log-likelihood turns out to be the least squares loss, the loss that is based on the square of the difference between the prediction and the target variable. In the case of classification, the assumption about the distribution of target variables is categorical distribution. The outputs of the network are converted into probabilities using softmax function, and negative log-likelihood turns out to be cross-entropy loss. This loss measures how much probability was assigned by the model to the correct class.

3. Fully connected layers connect every input to every neuron, leading to many parameters when working with images. For instance, an image of dimensions (224 \times 224 \times 3) RGB will have over 150,000 inputs, and one output in a fully connected layer will have over 150,000 parameters. On the other hand, a convolution layer uses a much smaller filter. A filter of size (3 \times 3) for an RGB image will have 27 parameters. Filters are used to process an image in different locations and therefore are known as weight sharing. Local connectivity means that units consider just a small portion of an image. A convolution operation is invariant to translation because, if the object moves on the image, then the detected feature will be moved too. Additional pooling layers can help achieve more translation invariance. Convolution layers can be added for considering bigger regions of an image.

I then asked Claude to find the gaps in my new responds and it gave me this:

These are the gaps I found, which are smaller than in the first round.

Q1: Deep vs. shallow

- You say a deep network "can create many more" regions but don't say how many. State the scaling: a shallow network's regions grow roughly linearly with the number of hidden units, while a deep network's can grow exponentially with depth. Without this, the central claim is still qualitative.
- You don't mention that the activation function must be nonlinear. Without it, stacked layers collapse into a single linear map and depth would give no benefit.
- "Harder to train" is vague. Naming vanishing or exploding gradients would make it concrete.
- Say that the efficiency advantage applies to certain families of functions, not all of them. Right now it could read as universal.

Q2: Maximum likelihood and losses

- You mention turning a product into a sum, but not why a product appears. It requires the assumption that training examples are independent (and identically distributed).
- You don't say that taking the log doesn't change the best parameters, because log is monotonic. You also don't mention the practical benefit of avoiding numerical underflow from multiplying many small probabilities.
- For regression, say what the model actually outputs: the network predicts the *parameter* (the mean) of the distribution, not the distribution itself.
- Explain why fixing the variance matters: the leftover terms are constants that don't affect the minimization, which is what leaves plain squared error.
- For the Bernoulli case, mention that a sigmoid produces the probability, as softmax does for the categorical case.

Q3: Fully connected vs. convolutional

- A 3×3 filter on an RGB image has 27 weights (plus a bias) for *one* output channel. A real layer uses many filters, so the comparison is clearer if you note this, or compare total parameters for layers with the same output size.
- Name the receptive field. You describe it ("sees larger parts of the image") but never use the term.
- Explain why fully connected layers suit images poorly beyond parameter count. Flattening discards spatial structure, and they have no built-in assumption that nearby pixels are related or that patterns repeat across positions.

- You could mention in passing that stride and padding control how the filter moves and the output size.

What changed from your first draft (for the "briefly explain what changed and why" part of step 4): you added concrete numbers, the shallow vs. deep tradeoff, the negative log-likelihood and its assumptions, the softmax and Bernoulli details, weight sharing and local connectivity, and the equivariance vs. invariance distinction.

Here is my explanation for each question:

Q1: Deep vs. Shallow Networks

I added more detail about how the number of linear regions grows in deep networks compared to shallow ones. I also explained that the activation functions need to be nonlinear for adding layers to provide a benefit. Finally, I mentioned vanishing and exploding gradients and clarified that deep networks are more efficient for certain types of functions, not necessarily every function.

Q2: Maximum Likelihood and Loss Functions

I added the assumption that the training examples are independent, which explains why their probabilities are multiplied together. I also explained why we use the log of the likelihood and how the Gaussian and Bernoulli/categorical assumptions lead to the different losses. These changes make it clearer how maximum likelihood connects to least squares and cross-entropy.

Q3: Fully Connected vs. Convolutional Layers

I added a more specific comparison of the number of parameters and explained that a real convolutional layer uses multiple filters. I also added the ideas of receptive fields and spatial structure to explain why convolutional layers work better for images. Finally, I clarified translation equivariance and explained that fully connected layers lose some of the useful spatial relationships when the image is flattened.

I learned how to better format my answer when it comes to answering questions about deep learning topics, I think I have a firm grasp on deep learning concepts but sometimes struggle with the details of what I should be explained and what's worth explaining and what's not because I would explain a bit too much and not give enough context of what the question is about but will branch out to things that shouldn't be covered. I strongly recommend people to use notebookLM, even though I used claude for this assignment,

notebookLM is a really good way to study because you can upload slides and videos to give the AI context on what you want to learn or study and it will explain it using the context you gave and this can generate practice quizzes, videos, flash cards and much more and its free. I use it to study in every class including this one. What I like most is you can upload an old quiz to give the AI context on how quizzes are formatted and what they cover so when you tell it to make a practice quiz, it will take the old quiz as an example.